

PolyWatch — Polymarket Anomaly Monitoring Dashboard

PolyWatch is a full-stack application for monitoring price anomalies in [Polymarket](#) prediction markets. This repository contains the **frontend**, which provides an interactive dashboard that displays market price trends and detected anomaly events in real time.

Note

This is a README file for directory PolyWatch/visualization.
(<https://github.com/wwp-max/PolyWatch/tree/main/visualization>).

What Does This Project Do?

Polymarket is a decentralized prediction market platform where users can trade on the outcomes of real-world events (such as election results, policy changes, etc.). Each market's price reflects the crowd's estimated probability of an event occurring — a price of 0.65 means the market believes there is a 65% chance the event will happen.

PolyWatch aims to **automatically detect anomalous price behavior** in these markets, such as:

Anomaly Type	Description
Z-Score Spike	Price suddenly deviates significantly from the historical mean (detected via statistical methods)
Whale Trade	A large transaction ("whale" trade) causes sharp price movements
Whale Directional Bias	Large transactions are concentrated in one direction, potentially indicating manipulation
Benford Violation	The digit distribution of price changes does not conform to Benford's Law, suggesting possible manipulation

The frontend dashboard allows you to:

- Switch between different prediction markets using the left sidebar
- View each market's price history chart (with zoom and drag support)
- View statistics (average price, number of data points, data time range)
- Browse the list of detected anomaly events (with severity levels and detailed parameters)

Tech Stack

Layer	Technology
Framework	Next.js 16 (App Router)
UI Library	React 19
Language	TypeScript 5
Styling	Tailwind CSS 4 + shadcn/ui
Charts	Apache ECharts 6 (via echarts-for-react)
Data Fetching	TanStack React Query 5
Icons	Lucide React
Package Manager	pnpm
Backend	Flask (Python)
Database	TimescaleDB (PostgreSQL extension for time-series data)
Containerization	Docker Compose (database + data collector)

Project Structure

visualization/	
├─ app/	# Next.js App Router pages
│ ├─ layout.tsx	# Root layout (global providers, fonts, metadata)
│ ├─ page.tsx	# Main dashboard page
│ ├─ globals.css	# Global styles + Tailwind theme variables
│ └─ favicon.ico	# Browser tab icon
├─ components/	# Frontend UI components
│ ├─ MarketSidebar.tsx	# Left sidebar market list (collapsible)
│ ├─ StatsBar.tsx	# Statistics cards (avg price, data count, time range)
│ ├─ PriceChart.tsx	# Price trend chart (ECharts, anomaly markers)
│ ├─ AnomalyFeed.tsx	# Anomaly event list (scrollable, categorized, detailed)
│ ├─ ThemeToggle.tsx	# Dark/light theme switch
│ └─ ui/	# shadcn/ui base components
│ ├─ badge.tsx	
│ ├─ button.tsx	
│ ├─ card.tsx	
│ ├─ scroll-area.tsx	
│ └─ separator.tsx	
├─ lib/	# Frontend core logic and shared modules
│ ├─ types.ts	# TypeScript types aligned with backend API
│ ├─ api.ts	# Pure fetch layer for backend endpoints
│ ├─ providers.tsx	# React Query QueryClientProvider wrapper
│ ├─ theme.tsx	# Theme context (dark/light mode)
│ ├─ utils.ts	# Utility helpers (e.g., className merging)
│ ├─ hooks/	
│ │ └─ index.ts	# React Query hooks (useMarkets/usePriceHistory/...)
│ ├─ services/	
│ │ └─ index.ts	# Deprecated legacy service layer (kept for reference)
│ └─ mock/	
│ └─ data.ts	# Historical mock data (no longer used by runtime)
├─ public/	# Static assets served by Next.js
│ ├─ file.svg	
│ ├─ globe.svg	
│ ├─ next.svg	
│ ├─ vercel.svg	
│ └─ window.svg	
├─ backend/	# Backend code (Flask API + pipeline + analysis)
├─ FILES/	# Archived planning artifacts for Milestone 2
├─ docs/	# Planning/design documents
├─ polywatch-frontend/	# Milestone 2 code archive
├─ README.md	# This document
├─ package.json	# Frontend dependencies and scripts
├─ pnpm-lock.yaml	# Locked Node dependency versions
├─ pnpm-workspace.yaml	# pnpm workspace config
├─ tsconfig.json	# TypeScript config
├─ next.config.ts	# Next.js config
├─ postcss.config.mjs	# PostCSS config
├─ eslint.config.mjs	# ESLint config
├─ components.json	# shadcn/ui config
├─ .gitignore	# Git ignore rules

```
|— findings.md           # Working notes from integration
|— progress.md          # Progress log
|— task_plan.md         # Task
```

Data Flow

The overall data flow of the application:

```
Polymarket API → Data Collector → TimescaleDB → Flask API → Frontend
(public quotes)  (Docker container) (Docker container) (Python)  (Next.js)
                  fetches every 5min  time-series store  :5001      :3000
```

Internal data flow within the frontend:

```
Flask API (:5001)
  |
  ▼
lib/api.ts          ← Pure fetch functions, no React dependency
  |
  ▼
lib/hooks/index.ts  ← React Query hooks, automatic caching/retry/loading states
  |
  ▼
Components (page.tsx, MarketSidebar, PriceChart, StatsBar, AnomalyFeed)
```

Step-by-Step Setup Guide

Note

You can run the entire front-end and back-end process in this directory (visualization, <https://github.com/www-max/PolyWatch/tree/main/visualization>).

Prerequisites

Make sure the following software is installed on your machine:

Software	Minimum Version	Check Command	Installation
Node.js	18+	node -v	nodejs.org or brew install node
pnpm	8+	pnpm -v	npm install -g pnpm
Python	3.9+	python3 --version	python.org or brew install python
Docker	20+	docker --version	docker.com
Docker Compose	2+	docker compose version	Included with Docker Desktop

Tip: macOS users are recommended to install the above tools via [Homebrew](https://brew.sh).

Step 1: Start the Database (TimescaleDB)

The database runs via Docker Compose with a pre-configured initialization script.

```
# Navigate to the data pipeline directory
cd backend/data_pipeline

# Start TimescaleDB (runs in background)
docker compose up -d timescaledb

# Wait for the database to be ready (look for "healthy" status)
docker compose ps
```

You should see output similar to:

NAME	STATUS	PORTS
data_pipeline-timescaledb-1	Up (healthy)	0.0.0.0:5433->5432/tcp

Database connection details (you typically don't need to connect manually, but just in case):

- Host: localhost
 - Port: 5433
 - Database: polywatch
 - Username: polywatch
 - Password: polywatch
-

Step 2: Import Data (If the Database Is Empty)

If your database is brand new, you need to import seed data. `init.sql` automatically creates the table schema, but price history data needs to be imported manually:

```
# Still in the backend/data_pipeline directory
# Import seed data (CSV)
docker compose exec timescaledb psql -U polywatch -d polywatch \
  -c "\COPY price_history(time, token_id, price) FROM '/docker-entrypoint-initdb.d/./price_history_seed.csv'
  CSV HEADER"
```

Alternatively, start the data collector to automatically fetch the latest data from Polymarket:

```
# Start the collector (fetches automatically every 5 minutes)
docker compose up -d collector
```

Step 3: Start the Backend API Server

```
# Return to the project root directory
cd ../../

# Create a Python virtual environment (first time only)
cd backend
python3 -m venv venv

# Activate the virtual environment
source venv/bin/activate      # macOS / Linux
# venv\Scripts\activate       # Windows

# Install Python dependencies (first time only)
pip install -r requirements.txt

# Start the API server
```

```
python -m core_analysis.api_server
```

You should see:

```
=====
PolyWatch API Server
http://localhost:5001
=====
```

Available endpoints:

```
GET /api/health          - Health check
GET /api/markets         - Market list
GET /api/markets/<slug>/prices - Price history
GET /api/markets/<slug>/stats - Market statistics
GET /api/markets/<slug>/gaps  - Data gaps
GET /api/anomalies       - Anomaly event list
POST /api/analyze/<slug>    - Real-time analysis
```

Verify the backend is working (open a new terminal window):

```
# Test health check
curl http://127.0.0.1:5001/api/health
# Should return: {"service":"polywatch-api","status":"ok"}

# Test market list
curl http://127.0.0.1:5001/api/markets
# Should return a JSON array containing multiple markets
```

Step 4: Start the Frontend

```
# Return to the project root directory (visualization, https://github.com/wwp-
max/PolyWatch/tree/main/visualization)
cd ..

# Install frontend dependencies (first time only)
pnpm install

# Verify the environment variable file exists
cat .env.local
# Should display: NEXT_PUBLIC_API_URL=http://127.0.0.1:5001
# If the file does not exist, create it manually:
# echo "NEXT_PUBLIC_API_URL=http://127.0.0.1:5001" > .env.local

# Start the development server
pnpm dev
```

Then open <http://localhost:3000> in your browser, and you should see the dashboard!

Quick Start Cheat Sheet

If you have already completed the initial setup, the daily startup only requires:

```
# Terminal 1: Database
cd backend/data_pipeline && docker compose up -d timescaledb

# Terminal 2: Backend API
cd backend && source venv/bin/activate && python -m core_analysis.api_server

# Terminal 3: Frontend
pnpm dev
```

Then open your browser and navigate to <http://localhost:3000>.

Backend API Reference

Overview of the APIs called by the frontend:

`GET /api/markets`

Returns the list of all markets.

```
[
  {
    "slug": "presidential-election-winner-2024",
    "question": "Will Donald Trump win the 2024 US Presidential Election?",
    "active": true,
    "lastPrice": 0.95,
    "prevPrice": 0.93
  }
]
```

`GET /api/markets/<slug>/prices`

Returns the price history for a specific market (sorted by time).

Parameter	Type	Description
since	query (optional)	ISO date string; only returns data after this date

```
[
  { "time": "2024-10-01T00:00:00+00:00", "price": 0.52 },
  { "time": "2024-10-01T01:00:00+00:00", "price": 0.53 }
]
```

`GET /api/markets/<slug>/stats`

Returns statistics for a specific market.

```
{
  "slug": "presidential-election-winner-2024",
  "question": "Will Donald Trump win...",
  "rowCount": 4320,
  "firstTime": "2024-10-01T00:00:00+00:00",
  "lastTime": "2024-11-06T12:00:00+00:00",
  "avgPrice": 0.5834
}
```

`GET /api/anomalies`

Returns the list of anomaly events.

Parameter	Type	Description
slug	query (optional)	Filter by market slug
severity	query (optional)	Filter by severity level: low / medium / high

```
[
  {
    "id": 1,
    "marketSlug": "presidential-election-winner-2024",
    "detectedAt": "2024-10-15T08:00:00+00:00",
    "eventType": "zscore_spike",
    "severity": "high",
    "detail": { "z_score": 3.2, "threshold": 2.5, "price": 0.68 }
  }
]
```

POST /api/analyze/<slug>

Runs real-time anomaly detection on a specific market (does not write to the database; only returns analysis results).

Troubleshooting

Frontend displays "Failed to load markets"

Cause: The frontend cannot connect to the backend API.

Checklist:

1. Is the backend API running? Check your terminal for Flask output.
2. Is the port correct? Verify that `.env.local` contains `http://127.0.0.1:5001`.
3. Test the backend with `curl http://127.0.0.1:5001/api/health`.

Backend reports a database connection error

Cause: TimescaleDB is not running or the port is incorrect.

```
# Check Docker container status
cd backend/data_pipeline && docker compose ps
```

```
# If the container is not running
docker compose up -d timescaledb
```

```
# Wait for it to reach "healthy" status
docker compose ps
```

Chart displays "No price data available"

Cause: There is no price data in the database for the selected market.

Verify whether data has been imported:

```
# Connect to the database and check
docker compose exec timescaledb psql -U polywatch -d polywatch \
  -c "SELECT slug, COUNT(*) FROM price_history JOIN markets USING (token_id) GROUP BY slug;"
```

pnpm install fails

Try clearing the cache and reinstalling:

```
rm -rf node_modules pnpm-lock.yaml
pnpm install
```

Development Guide

Build for Production

```
pnpm build    # Compile + type check + optimize
pnpm start    # Start the production server
```

Type Checking

```
npx tsc --noEmit
```

Linting

```
pnpm lint
```

Change the Backend API URL

Edit the `.env.local` file in the project root:

```
NEXT_PUBLIC_API_URL=http://127.0.0.1:5001
```

You need to restart the frontend development server (`pnpm dev`) after making changes.

License

This project is a CityU CS6290 course project, intended for academic use only.